COPPER-LINE: Committed Pointer-Provenance as Prefetch Authority for Safe Data-Memory-Dependent Prefetching

Full research draft with ARM/AArch64 gem5 SE, ARM64 full-system CTLW timing ROI, Vivado RTL, and prior-art review

Status: full paper draft, 8-page target when formatted with standard architecture-paper tables and figures.

Abstract

Data-memory-dependent prefetchers (DMPs) improve performance on irregular pointer-heavy workloads by using memory contents as future addresses. Recent attacks, including Augury and GoFetch, show that this optimization can violate constant-time assumptions: data that merely resembles a pointer may trigger secret-dependent cache activity even when the program never architecturally uses that data as an address. This paper proposes COPPER, a committed pointer-provenance mechanism for DMPs. COPPER changes the authority model of data-driven prefetching. Instead of allowing the prefetcher to act on address-shaped data, it permits DMP dereference only when the exact source cache word has previously been proven by committed architectural execution to be a pointer source and has remained clean since that proof. Writes, fills, invalidations, domain mismatches, and failed translation or permission checks destroy or block the proof; a line-resident implementation also clears on eviction, while a bounded proof ledger may retain clean proofs across replacement.

The refined mechanism adds Recursive Carried-Provenance (RCP) runahead: prefetched pointer lines may seed further DMP requests only when the prefetched source word is already covered by committed provenance in the ledger. A Committed Page-Translation Queue (CPTQ) extends proof-gated issue across pages only after a valid process translation is observed. A Commit-Epoch Provenance Filter (CEPF) guards the backend proof path so stale in-flight source tags cannot recreate proof after a source-word overwrite. Full-system testing exposed two further requirements: Provenance Address-Space Binding (PASB), which keys source proofs by an address-space token rather than a hardware context alone, and Committed Target-Line Witnessing (CTLW), which permits cross-page recursive target formation only from an exact committed virtual-to-physical line witness and makes witness-derived fills terminal until demand-validated. We evaluate COPPER using adversarial trace simulation, graph-style provenance traces, SystemVerilog RTL synthesized in Vivado, ARM/AArch64 gem5 syscall-emulation prototypes, and an ARM64 full-system Linux path. On a mixed benign/adversarial trace, a naive DMP achieves 3.628x speedup but performs 2048 data-at-rest prefetches, 1076 cross-domain prefetches, and 2616 unproven-line prefetches; COPPER-LINE achieves 2.414x speedup while eliminating modeled unsafe dereferences. On a ten-seed CSR-like graph trace, COPPER-epoch gives 3.276x speedup while blocking data-at-rest and stale rewritten-edge prefetches. A GAPBS-backed topology trace over five generated Kronecker graphs shows COPPER-epoch and a compressed line-provenance directory (CLPD) retain 1.770x and 1.896x speedup, respectively, with zero data-at-rest, unproven-edge, or stale-slot prefetches; CLPD also closes a proof-capacity cliff by recovering a 2.115x g12 edge-scan speedup with 8192 line entries where the edge-exact ledger needs 131072 entries for 2.369x. An expanded GAPBS-style kernel sensitivity sweep over 4,320 graph/kernel/table/cache/lookahead runs preserves zero COPPER unsafe modeled prefetches, while naive DMP produces 81,605,320 unsafe modeled prefetches and source-only provenance still produces 284,488. Under an explicit storage model, CLPD gives about 30.86-32.00x full-coverage proof-storage reduction on those graphs; on g12, 8192 CLPD entries cost 54 KiB versus 1696 KiB for the edge-exact capacity point that reaches 2.369x. In gem5 pointer-chain workloads, recursive COPPER improves ARM32 page-permuted lists by 6.76-6.78% and random lists by 5.59-5.66%; hand-generated AArch64 SE binaries show 6.77% and 5.61% on the same full-list shapes. AArch64 CPU-model sensitivity remains positive on Minor and O3 cores: 2.64-2.79% for Minor and 2.68-2.77% for O3, while stride remains at 0.11-0.59% on those irregular shapes. The full-system path boots ARM64 Ubuntu/Linux 6.8.12, executes native static AArch64 ROIs under timing CPU after atomic boot, and attaches COPPER inside the L1D cache hierarchy. On a tiny ROI, naive pointer-shaped DMP produces 30 translation-faulted recursive attempts, pre-PASB COPPER still permits 5, and PASB-COPPER reduces this to zero. On larger full-system generated pointer ROIs, CTLW-terminal removes the remaining PASB-only translation faults and gives small positive timing movement: -0.531% ticks on page-permuted pointers and -0.271% on random pointers versus no prefetch, while blocking about 15k unproven pointer-shaped candidates per run. A separate full-system AArch64 graph-gather binary with stable CSR-like edge slots gives COPPER-CTLW a -0.367% tick movement versus no prefetch, blocks 8,660 unproven candidates, and records zero translation faults; stride still wins on this graph binary because its edge array is sequential. We also install LLVM/clang and compile freestanding AArch64 C suites. A graph/hash/tree/fake-pointer suite shows COPPER blocks 679 unproven candidates with zero translation faults but is 0.093% slower than no prefetch. A GAPBS-inspired BFS/SSSP/PageRank/CC mini-suite shows COPPER blocks 952 unproven candidates, eliminates 408 CTLW misses and 408 unavailable recursive translations seen by naive DMP+CTLW, records 7,729 terminal stops, and matches no-prefetch ROI ticks; a larger 1024-node timing-mode rerun blocks 1,340 unproven candidates, removes naive's 6 CTLW misses and 6 unavailable translations, records 50,737 terminal stops, and gives a small -0.208% tick movement versus no prefetch. Stride remains faster on these mini-suites because their edge arrays are sequential. Two bounded state-space checkers pass the full rule and find short counterexamples for weakened variants; the richer checker explores 11,419 reachable states to depth 12 while modeling source value/epoch, CEPF, PASB, exact CTLW, witness invalidation, and terminal fills. A Vivado 2025.2 synthesis of the core line-provenance RTL on an Artix-7 target meets a 10 ns constraint with +8.122 ns slack and uses no BRAM or DSPs; the CEPF bridge uses 5 LUTs, and a fresh 10-script Vivado authority regression passes with no failures. To the best of public knowledge, COPPER is the first public DMP defense to make committed pointer provenance, address-space binding, and committed target-line translation witnesses the hardware authority for recursive data-driven prefetch.

CS-SARI, a conflict-scoped refinement of SARI, adds candidate-specific SoC revocation evidence. Its wired SARI-to-CLPD/CTLW/full-authority RTL harness passes 12 directed plus 10,000 randomized XSIM samples with conflict_hold=1245, avoided_global_hold=1007, and errors=0. A GAPBS-topology revocation proxy over graph-derived source and target locality reports 82.06% aggregate hold reduction versus global SARI hold, 269,879 authorized candidate opportunities recovered, zero CS-SARI modeled unsafe issues, and 59,013 unsafe issues for a no-hold policy. A bounded composition checker explores 7,555 reachable states for the full CLPD-plus-CTLW-plus-CS-SARI rule, proves no stale source/target authorization in the model, and shows weakened variants fail for incoming source revocation, queued source revocation, target remap, token TLBI, and global TLBI hazards. A 20-configuration queue-depth/conflict sweep keeps CS-SARI unsafe issues at zero while a no-hold control produces 1,649,883 unsafe modeled issues and the median scoped-hold reduction is 72.06%.

A 2026-06-15 full-system refresh adds a stronger AArch64/Linux evidence point. A ROI-bracketed heap-pointer workload with 32,768 heap nodes, 16 traversal passes, 32,768 fake pointer-shaped words, rewrites, and checksum validation shows CLPD-64K improves all three heap-layout seeds with mean -2.866% ROI ticks versus no-prefetch while naive DMP slows them by mean +15.214%. A provenance epoch boundary (PEB) then removes the remaining fake-only warm-state leakage: CLPD-64K+PEB blocks 131,066 of 131,066 fake pointer-shaped observations, issues zero prefetches, drops 76,560 stale authority entries, and is only +0.033% versus no-prefetch. The same PEB mode still improves all three heap seeds with mean -2.905%, zero CTLW misses, zero translation faults, and matching checksums. On official AArch64 GAPBS BFS/CC/PR/SSSP/BC/TC at g10, CLPD-64K+PEB runs all six kernels with rc=0, +0.015% aggregate timing versus no-prefetch, zero translation faults, zero proof evictions, and 340,128 pre-boundary authority entries dropped. A scalable true-dual-port CLPD SRAM directory passes 18 directed plus 4,000 randomized XSIM tests; the 64K-entry configuration used by gem5 synthesizes on xc7a200tfbg676-2 with 629 LUTs, 156 FFs, 260 BRAM tiles, and WNS 3.274 ns at 10 ns, then routes out-of-context with WNS 0.362 ns. PEB itself is a small per-domain epoch/token RTL block: XSIM covers stale-boundary, token, domain-isolation, and wrap-block cases with errors=0, and Artix-7 synthesis uses 346 LUTs and 147 FFs with WNS 3.782 ns. PEB is not novel because it uses an epoch; the narrower contribution is binding DMP proof authority to a provenance epoch and measuring the containment/performance result.

A later same-day public-workload refresh adds Olden AArch64 full-system results and stronger conventional prefetch baselines. On randomized-allocation Olden, stride slows by +10.107% on the small suite and +11.565% on a medium Treeadd/Bisort/Health subset. Naive DMP is near neutral on the small suite (+0.039%) and improves the medium subset by -2.829%, but produces 188,223 and 123,516 CTLW misses, respectively. COPPER CLPD-64K+PEB improves by -0.398% on small Olden and -2.616% on medium Olden while cutting those CTLW misses to 29,039 and 47,145, blocking 320,013 and 185,023 unproven candidates, and preserving zero translation faults. A validation-only Bisort build emits identical baseline/COPPER count, checksum, and histogram fingerprints for initial, forward-sort, and backward-sort phases. Built-in gem5 prefetchers are faster on Olden: DCPT reaches -5.742% small / -7.025% medium, SPP reaches -2.963% small / -5.870% medium, and AMPM reaches -2.465% small / -3.909% medium. Indirect-memory and ISB are weaker but still useful controls. These are conventional address-correlation baselines, not content-derived DMP safety baselines. They force the paper's claim to be precise: COPPER is a safe authority mechanism for data-derived pointer prefetching, not a replacement for the best address-stream prefetcher on every workload.

A 2026-06-16/17 public-application stress refresh adds medium/stress full-system AArch64 runs for SQLite, Lua, Duktape, and yyjson with none, stride, naive, copper_clpd64k_peb, dcpt, spp, ampm, and spp_copper_slack policies. Across the eight medium/stress app points, standalone COPPER is faster than naive DMP on 4/8 and has fewer L1D demand misses on 6/8; it averages -0.679% ticks versus no prefetching, +1.140% memory-bus bytes, and 90.3% fewer CTLW misses than naive DMP, with zero translation faults and zero target-witness evictions. SPP is the best ordinary address-stream baseline on all eight points, with mean -15.337% ticks. SPP+COPPER slack tracks that baseline at mean -15.344%, with an average signed gap of -0.006 percentage points and a 0.360-point worst absolute gap, while preserving COPPER child-filter activity, zero translation faults, and 92.9% CTLW reduction versus naive DMP. A follow-on repeated public-app portfolio covers SQLite, Lua, and Duktape with three medium layout seeds and two stress layout seeds per engine. Across 15 engine-seed points and 75 policy rows, all checksums match per point, all runs return rc=0, translation faults remain zero, standalone COPPER beats unsafe naive DMP on 9/15 points, COPPER cuts aggregate naive CTLW misses by 90.706%, and SPP+COPPER slack stays within 0.760 percentage points of SPP while cutting CTLW misses by 91.505%. A new energy/pollution proxy scorecard over gem5 counters gives standalone COPPER a 2.105% mean pressure score versus 2.364% for naive DMP, a 10.9% lower proxy pollution score, and shows lower-or-equal bus bytes and DRAM reads on 7/8 points; SCOOP's slack companion is within 0.5% runtime of SPP on all eight points while adding 0.071 bus-byte-delta points and 0.628 pressure-score points over SPP on average. A bounded OoO-LSQ proof-contract checker then targets the backend integration objection: the full contract has a reachable legal proof witness and zero unsafe bounded proof creations, while execute-stage proof, unretired-source proof, missed flush clearing, missed source revocation, missing CEPF epoch/value, and missing translation/permission gates all produce short counterexamples. A separate TLB/coherence authority-contract checker explores 39,098 full-contract states and covers source revocation, queued target revocation, remap, token TLBI, global TLBI, and permission-downgrade hazards; the full rule passes, while page-level witness, source-only authority, missed queue hold, missed TLBI/remap clearing, and missed permission-gate variants fail. A matching issue-side TLB/coherence authority filter in RTL passes 27 directed plus 10,000 randomized XSim checks and synthesizes on Artix-7 with 332 LUTs, 167 FFs, no BRAM/DSP, and WNS +6.898 ns at 10 ns. This separates the low-overhead authority path (standalone COPPER) from the performance/security coexistence path (SCOOP-style slack companion), and it keeps the claim honest: COPPER is not a universal replacement for conventional address-correlation prefetchers.

1. Introduction

Modern out-of-order CPUs rely on prefetching to hide memory latency. Conventional prefetchers infer future misses from the address stream: strides, deltas, temporal recurrence, or instruction correlations. Data-memory-dependent prefetchers go further. They inspect memory data and use pointer-like values as prefetch targets, making them attractive for linked lists, graph traversal, hash tables, and arrays of pointers. This optimization is natural from a performance viewpoint: when useful addresses are stored in memory, address-stream-only prediction can be late or blind.

The security problem is that DMPs blur the separation between data and addresses. Constant-time cryptographic software is normally written so secret data does not influence branches or memory addresses. DMPs can violate that contract under the software: a value sitting in memory may resemble an address and cause a prefetch, creating observable cache state. Augury showed that Apple processors contain an Array-of-Pointers DMP that can leak data at rest. GoFetch then showed practical key extraction attacks against constant-time cryptographic implementations by exploiting DMP activation on pointer-looking values.

Existing mitigations tend to be coarse or software-specific. A platform may disable a DMP under a timing-independence mode or during sensitive scheduling windows. A compiler may transform secrets so they do not look like addresses. These are valuable, but they do not answer the architectural question: can a DMP retain useful pointer prefetching while refusing to grant prefetch authority to arbitrary data?

COPPER-LINE answers that question with a narrow hardware invariant:

```
A DMP may dereference a memory word only if committed execution has already
used that exact cache word as a pointer source, and the word has remained clean
since the proof was created.
```

This is not a claim that metadata, cache bits, taint tracking, or pointer prefetching are new. They are not. The contribution is the authority rule for a DMP and its recursive consequence. Address-shapedness is no longer sufficient. A prefetched line is also not sufficient by itself. Every dereference source, including sources reached by prior prefetches, must be backed by committed pointer provenance.

The paper makes four contributions:

- It defines the COPPER-LINE invariant: no DMP dereference from unproven, stale, cross-domain, or permission-invalid data words.

- It defines Recursive Carried-Provenance runahead: a DMP prefetch fill may become a new source only if the source word/value already has committed provenance in the ledger.

- It gives an RTL-realizable metadata lifecycle: committed pointer use sets a per-word proof bit; writes, fills, invalidations, and coherence events clear proof; translation and domain checks gate use.

- It adds CLPD and PEB as scalable authority-management structures: CLPD compresses retained source proof by cache line, and PEB invalidates pre-boundary proof authority with a per-domain epoch/token salt rather than a directory sweep.

- It evaluates the security/performance tradeoff with trace simulation, Vivado RTL, ARM/AArch64 gem5 syscall-emulation pointer workloads, and ARM64 full-system Linux/native-AArch64 execution with the prefetcher attached in the cache hierarchy, including ROI-bracketed heap-pointer, fake-only, official GAPBS, and CTLW-terminal larger-workload timing.

2. Background and Motivation

2.1 Data-Memory-Dependent Prefetching

A DMP observes memory data and predicts future addresses from that data. In an array of pointers, for example, a conventional prefetcher can predict future accesses to A[i], but a DMP can prefetch *A[i]. This can help irregular workloads where addresses have weak spatial or temporal recurrence.

The same behavior creates a side channel. If the DMP treats any address-shaped value as a candidate pointer, then the microarchitecture performs a memory action based on data that software may have intentionally kept out of address generation. The resulting cache state can be measured by an attacker.

2.2 Why Disable Is Unsatisfying

Disabling DMPs is a reasonable emergency mitigation, especially during cryptographic code. It is also architecturally blunt. Pointer-intensive code can lose useful prefetching, and mixed workloads may contain security-sensitive and ordinary pointer-heavy regions interleaved at fine granularity. A more satisfying mechanism lets safe pointer prefetches continue while blocking unproven data-driven dereferences.

2.3 The Authority Mistake

The core mistake is treating "looks like an address" as authority. A 64-bit value can fall into a canonical address range by accident, attacker shaping, or cryptographic intermediate manipulation. That value should not authorize the memory system to fetch through it. In ordinary architectural execution, a load address has authority because an instruction committed using that address under translation and permission checks. COPPER-LINE imports that idea into the DMP path.

3. Threat Model and Security Goal

We consider an attacker who can observe cache timing and can influence victim data or co-located memory state enough to create pointer-shaped secret-dependent values. The attacker may attempt to trigger the DMP on memory words that the victim has not architecturally used as addresses. The attack target is DMP-created cache activity that reveals data-at-rest, cross-domain values, stale overwritten values, or permission-invalid targets.

COPPER-LINE does not attempt to prevent every cache side channel. It does not hide legitimate architectural memory accesses. It does not make all software constant-time. It does not protect a program from its own committed secret-dependent address generation. Its goal is narrower:

```
The DMP must not create a dereference prefetch from a source word that lacks
clean committed pointer provenance in the current permitted domain.
```

This goal is intentionally narrow enough to be implementable in cache metadata and broad enough to cover the DMP failure mode exposed by Augury and GoFetch.

4. COPPER-LINE Mechanism

4.1 Metadata

COPPER-LINE stores provenance alongside cache-line metadata or in an equivalent source-line proof ledger. For a 64-byte line with eight 64-bit words, the minimal line-resident state is one proof bit per word:

```
proof_bits[line][word] = 1 if the current word contents have been proven
                         by committed execution as an address source.
```

A small domain color may be stored per line. Depending on the target core, this color could represent a security state, ASID/VMID-derived context, privilege partition, or implementation-defined protection context. The domain color is not meant to replace full translation; it is an early guard. Normal translation and permission checks still gate the prefetch target.

4.2 Proof Creation

Proof is created only on committed architectural use:

```
if demand memory operation commits
and its effective address was sourced from cache word (L, W)
and translation/permission succeeded:
    proof_bits[L][W] = 1
    line_domain[L] = current_domain
```

The mechanism requires the backend or load/store queue to identify when a committed memory operation's address was generated from a loaded source word. In a research prototype, this can be modeled by instrumentation or compiler trace annotations. In hardware, it can be implemented by carrying source-word tags through dependent address generation for the subset of loads that feed memory addresses. This is a metadata path, not a new architectural instruction.

4.2.1 Commit-Epoch Provenance Filter

A backend path must also avoid stale source tags. Suppose a load reads a pointer source word, a dependent memory operation carries that source tag, and a store overwrites the source word before the dependent operation commits. If the backend blindly emitted proof at commit, it could recreate provenance for a word that no longer contains the proven value. COPPER fixes this with a Commit-Epoch Provenance Filter (CEPF):

```
when a source word tag is captured:
    carry {line, word, domain, source_epoch}

on write/fill/invalidate of that source word or line:
    increment or invalidate source_epoch

on dependent memory op commit:
    create proof only if no squash/exception,
    translation and permission succeeded,
    and carried source_epoch == current source_epoch
```

CEPF is not a performance prefetcher. It is a proof-path invariant: committed pointer use is necessary but not sufficient if the source word changed while the dependent operation was in flight.

4.3 Proof Destruction

Proof is destroyed whenever the source word may no longer contain the proven pointer:

```
on store/write to word (L, W): proof_bits[L][W] = 0
on line fill/invalidation for L: proof_bits[L][*] = 0
on coherence invalidation/update for L: proof_bits[L][*] = 0
on DMA or external write visible to cache: proof_bits[L][*] = 0
```

The destruction rule is what makes COPPER-LINE different from a loose stream heuristic. A word is eligible only while it is clean since committed proof. In the smallest line-resident RTL, replacement also drops the proof because the metadata leaves with the cache line. In the gem5 model, clean proofs may be retained in a bounded source-line/value ledger across replacement; this improves warmup behavior but requires invalidation hooks for writes, coherence, DMA, and permission changes.

4.3.1 Compressed Line-Provenance Directory

Long graph scans exposed a storage cliff for an edge-exact proof ledger: if the ledger cannot retain the whole repeated edge stream, scan-order reuse can evict proofs before they are useful. COPPER-CLPD fixes this representation problem without weakening the authority rule. The Compressed Line-Provenance Directory stores one retained entry per source cache line, a per-word proof mask, and a source-line epoch:

```
clpd[source_line] = {source_line_epoch, proven_word_mask, context}
```

On a committed pointer use, CLPD sets the corresponding word bit under the current line epoch. On any write, fill, invalidation, DMA/coherence update, or permission-relevant event for that source line, the line epoch changes and the old directory entry stops authorizing all words in the line. The directory therefore compresses adjacent CSR or pointer-array proofs while staying conservative: a write to one word invalidates the whole retained line proof until demand execution recreates it.

4.4 DMP Gate

The DMP is allowed to issue a dereference prefetch only if:

```
allow =
    dmp_seed_valid
    and proof_bits[source_line][source_word]
    and line_domain[source_line] == source_domain
    and source_domain == target_domain
    and translation_ok
    and permission_ok
```

The DMP may still use ordinary address-stream prefetching outside this path. COPPER-LINE gates only dereference-style prefetches whose target comes from memory contents.

4.5 Why This Is Not Just a Combined RTL Block

The novelty is not "a bit and an AND gate." The bit and gate are an implementation of a new DMP authority invariant. A naive DMP grants authority to values that look like addresses. COPPER-LINE grants authority only to a source word whose current contents are covered by committed pointer provenance.

This differs from taint tracking. Taint tracking usually marks data as sensitive or untrusted and propagates that label through computation. COPPER-LINE marks a positive, narrow permission: this exact clean word may serve as a DMP source. It also differs from CHERI or MTE. Those mechanisms protect architectural pointer use or memory safety. COPPER-LINE protects a microarchitectural prefetcher that can otherwise act on data never used architecturally as an address.

4.6 Recursive Carried-Provenance Runahead

The initial COPPER-LINE rule is intentionally conservative: a line fill can propose a pointer candidate, but the DMP may issue only if that source word has committed proof. This creates a useful one-hop prefetch, but it does not by itself solve pointer-chain latency because the prefetched target line may contain the next pointer.

The refined mechanism adds Recursive Carried-Provenance (RCP) runahead:

```
on COPPER-issued prefetch target T from proven source S:
    record carried_provenance[T.phys_line] =
        {T.virt_line, requestor, context, addr_space_token, secure_state}

on fill of T:
    if T.phys_line matches carried_provenance
```

```
    and ledger_has_committed_proof(T.word, T.value, context, addr_space_token)
    and T.word is clean since proof:
        allow T.word to seed the next DMP candidate
    else:
        block recursive issue
```

RCP is not ordinary recursive pointer chasing. The prefetcher is not allowed to trust a value merely because it arrived through a prior prefetch. The carried record only preserves the identity and protection context of the prefetched line. Authority still comes from an existing committed proof for the exact source word/value. This distinction matters: it prevents unbounded speculative dereference through attacker-shaped memory while allowing repeated pointer structures to overlap multiple cache misses after the application has proven the structure once.

4.7 Committed Page-Translation Queue

Same-page prefetch issue is easy because the target page is already implied by the source translation. Irregular pointer structures, however, often cross page boundaries. COPPER therefore uses a Committed Page-Translation Queue (CPTQ) for cross-page targets:

```
if source word has committed provenance
and candidate target crosses page:
    enqueue candidate with source context
    issue only if process page table translation succeeds
    drop on failed or unavailable translation
```

CPTQ is deliberately not a general address oracle. It is reached only after the committed-provenance gate. In the gem5 prototype, syscall-emulation runs use the process page table. The first full-system AArch64 implementation used the core MMU functional translation path; the final full-system refinement below replaces recursive fresh translation with a committed target-line witness for cross-page recursive fills. A production AArch64-style core would map these checks to the normal prefetch translation and invalidation machinery, preserving ASID/VMID, security state, privilege, and permission checks.

4.8 Provenance Address-Space Binding

The first full-system COPPER integration found a subtle but important weakness: a hardware thread context is not the same as an address space. Linux can run different processes on the same CPU context, and low virtual addresses may be valid for one process but invalid for another. A proof table keyed only by source line, word, requestor, hardware context, and security state can therefore authorize a recursive candidate after the address-space identity has changed.

COPPER-PASB fixes this by adding a Provenance Address-Space Binding token to every source proof and carried-provenance record:

```
proof_key =
    {source_phys_line, source_word, requestor, secure_state,
     context_id, addr_space_token, value_token}
```

In the gem5 AArch64 prototype, addr_space_token is a compact token derived from the active translation identity (TTBR0_EL1, TTBR1_EL1, and CONTEXTIDR_EL1) when the source request carries a context. Other ARM/AMBA-connected systems could derive the token from ASID, VMID, security state, realm/world state, PASID, or an implementation protection color. The token is not an authority by itself; it only prevents a committed proof from moving across address spaces. Authority still requires clean committed source-word provenance and a successful target translation.

4.9 Committed Target-Line Witnessing

PASB fixes source-proof reuse across address spaces, but the larger full-system runs exposed a second issue: even an address-space-bound recursive source can ask the MMU to translate a candidate at an awkward time, after the original demand context has moved through kernel code or process teardown. Treating every recursive target as a fresh speculative translation either leaves faulted translation attempts in the evidence path or, if replaced with a page-granular witness, over-opens recursion and pollutes the cache.

COPPER-CTLW fixes this with an exact committed translation witness:

```
target_witness =
    {target_virt_line, target_phys_line, requestor, secure_state,
     context_id, addr_space_token}
```

On every committed demand access, the prefetcher records the virtual cache line and physical cache line under the current PASB token. A cross-page recursive prefetch from a prefetched line may form its physical target only if the exact target virtual line has a matching committed witness. The prefetch then uses the witnessed physical line directly instead of invoking a new speculative MMU translation. CTLW-derived fills are terminal: they may fetch data, but may not recursively seed another DMP request until a later demand access validates the line and creates ordinary provenance. This terminal rule was necessary because a page-level witness removed translation faults but amplified recursion to 85,377 prefetches and slowed the larger page-permuted ROI by 3.264%.

5. Security Argument

The core security argument is an invariant over the metadata state.

Invariant:

```
For any DMP dereference prefetch issued from source word (L, W), the current
contents of (L, W) have been used as an address source by committed execution
since the last write, fill, invalidation, or other data-changing event for L/W, and the
prefetch target passed domain, translation, and permission checks.
```

Proof sketch:

- Initially, all proof bits are zero, so no source word is eligible.

- The only transition that sets a proof bit is committed architectural pointer use.

- Any transition that may change the source word clears the proof bit. A line-resident implementation also clears on replacement; a retained-ledger implementation must keep an invalidation path for any coherence or external write to that source line.

- The DMP gate checks the proof bit, source/target domain match, translation, and permission.

- Therefore, a DMP dereference cannot be issued from an unproven, stale, cross-domain, or permission-invalid source word.

This proof does not require knowing a proprietary DMP's internal pattern detector. COPPER-LINE sits at the authority boundary: whatever heuristic proposes a candidate, the dereference cannot issue unless the source word has clean committed pointer provenance.

6. Evaluation Methodology

We built eight classes of artifacts.

First, a trace generator creates synthetic pointer-heavy workloads mixed with adversarial DMP candidates. The main trace contains benign linked pointer chains, data-at-rest secret-like values, cross-domain pointer-shaped values, and rewritten source words. The adversarial trace directly tests first-use, stale rewrite, cross-domain, translation failure, and permission failure cases.

Second, a trace simulator compares five policies:

Policy	Description
disabled	No DMP dereference prefetching.
naive	Any DMP candidate may prefetch if address-shaped.
copper_value	A global value-provenance table must match source, word, value, and domain.
copper_line	COPPER-LINE line/word proof must be clean and consistent.
copper_stream	Optional stream-certified policy with dirty-source blocking.

Third, SystemVerilog RTL implements the core line-provenance metadata gate, two earlier/optional gates, a full-authority CEPF/PASB/CTLW predicate gate, and a CLPD source-line proof directory. Vivado 2025.2 simulation verifies directed behavior, and randomized scoreboard tests check the core allow/block invariant across mixed commit, write, fill, invalidation, domain, translation, permission, source-epoch, address-space-token, witness, terminal-source, CLPD line-epoch, and directory-collision events. Vivado synthesis targets xc7a35tcpg236-1 with a 10 ns clock constraint for the blocks whose batch synthesis completed.

Fourth, a CEPF RTL bridge models the backend commit/proof path. It allows proof creation only for committed dependent memory operations with no squash, exception, translation failure, permission failure, or source-epoch mismatch.

Fourth-a, bounded finite-state invariant checkers explore the COPPER authority state machine. They are not full industrial formal signoff, but they check source cleanliness, committed proof, address-space token match, same-page/cross-page target state, exact target-line witness state, terminal witness-derived fill state, and in the richer checker, source value/epoch, CEPF-like in-flight backend source tags, witness invalidation, and proof soundness. The first checker has no counterexample within depth 10 for the PASB/CTLW/terminal rule. The richer checker explores 11,419 reachable states to depth 12 for the full authority rule. Intentionally weakened variants fail with short counterexamples.

Fourth-b, an assertion-focused SystemVerilog harness states the full-authority RTL predicate as SVA properties. It checks that every allowed DMP seed has exact committed source proof, PASB token match, non-terminal source status, target authority, and permission success, and that block-reason outputs match the named unsafe classes.

Fourth-c, an end-to-end CEPF-to-line SVA harness connects the commit-epoch proof bridge directly to the line-provenance DMP gate. It checks the multi-cycle path from commit-filtered proof creation through cache-line proof storage to DMP allow/block, including stale epoch blocking and write/fill/invalidate clearing.

Fourth-d, a standalone CTLW witness directory models the target-line witness object that the full-authority gate consumes. The RTL records exact virtual-line/physical-line/token witnesses and clears them on remap, token TLBI, global TLBI, or direct-mapped collision. Its testbench checks that page-level aliases and stale-token witnesses cannot authorize recursive cross-page issue.

Fourth-e, a CTLW-to-full-authority integration harness connects that witness directory to the combined allow/block gate. It checks that an exact live witness opens cross-page DMP issue, while no-witness, token-mismatch, same-index wrong-line, remap-cleared, TLBI-cleared, collision-evicted, terminal-source, permission-failure, and stale-source cases all block at the final authority predicate.

Fourth-f, a CLPD-to-CTLW-to-authority integration harness then connects the compressed source-line proof directory, the exact target-line witness directory, and the final authority gate. It checks that cross-page issue requires both a live compressed source proof and a live exact target witness, and that source-side write/fill/invalidate events plus target-side remap/TLBI events revoke the joint authority.

Fourth-g, SARI, a SoC Authority Revocation Interface, models the boundary between AMBA/CHI-style coherence events and COPPER metadata. It queues DMA/CHI/coherent-I/O source-line revocations, forwards target remap and TLBI events to CTLW, exposes ready/overflow status, and immediately holds DMP issue while revocations are incoming or draining.

Fourth-h, CS-SARI, Conflict-Scoped SARI, refines the hold rule so unrelated revocations do not globally stop safe DMP issue. A candidate is held only when its source line, target line, or address-space token conflicts with an incoming or queued revocation; overflow falls back to conservative global hold because the precise pending revocation set is no longer known.

Fifth, graph-style provenance traces use CSR-like edge slots. Repeated passes over graph edges create committed source-slot proof, an adversarial side array injects pointer-shaped data-at-rest values, and edge slots are rewritten after warmup to test stale source behavior. A second evaluator parses public GAPBS serialized .sg graphs and replays edge-scan and BFS-replay streams over the actual generated topology to test proof capacity and CLPD compression.

Sixth, we installed and ran free external tools. ChampSim was built locally and run on three synthetic trace shapes with no prefetching, next-line L1D prefetching, and IP-stride L1D prefetching. GAPBS was built locally and run with verification enabled on generated graph workloads. These runs are supporting evidence for baseline behavior and workload readiness; they do not carry COPPER provenance semantics.

Seventh, we integrated a CopperPrefetcher into gem5's cache prefetcher framework and ran ARM/AArch64 syscall-emulation pointer-chain binaries with private L1I/L1D caches, an L2, and DDR3 timing memory. The main ARM32 and AArch64 runs use ArmTimingSimpleCPU; a follow-on AArch64 sensitivity pass repeats the irregular full-list runs on ArmMinorCPU and ArmO3CPU. The gem5 model includes a committed proof ledger, clean-proof retention across replacement, CPTQ cross-page issue, and recursive carried-provenance runahead. We compare no prefetching, stride prefetching, a naive address-shapedness policy, one-hop COPPER with CPTQ, and recursive COPPER.

Eighth, we downloaded the public gem5 ARM64 Ubuntu 24.04 resources and ran a full-system Linux path with Linux 6.8.12, the ARM64 bootloader, a two-core ARM board, and no_systemd=true to avoid spending the local Windows run budget on ordinary service startup. The first run is an environment-validation boot/readfile probe: it checks that ARM64 Linux boots, mounts the disk image, loads the gem5 bridge, reads a host-supplied script, reports aarch64, and exits cleanly. The second run injects a static native AArch64 ELF, resets gem5 statistics immediately before the binary, runs the workload inside the guest, prints COPPER policy outputs, dumps ROI statistics, and exits. Follow-on timing passes attach none, stride, naive pointer-shaped DMP, pre-PASB COPPER, PASB-COPPER, and CTLW-terminal COPPER directly in the full-system L1D cache hierarchy. The final larger runs use generated page-permuted and random AArch64 pointer binaries with 8192 64-byte nodes, four chase passes, and a 4096-entry fake-pointer scan, plus an AArch64 graph-gather binary with stable CSR-like edge slots, repeated passes, random target nodes, a compute gap for prefetch lead time, and a fake pointer-shaped side array. We then install LLVM/clang 22.1.7 and LLD and compile freestanding C AArch64 suites: one graph/hash/tree/fake-pointer control and one GAPBS-inspired mini-suite with BFS, SSSP-like relaxation, PageRank-style gather, and connected-components-style propagation over pointer-valued adjacency arrays. These runs validate full-system guest execution, workload plumbing, cache-path integration, address-space-bound proof, committed target-line translation witnesses, compiled AArch64 workload support, and the remaining gap to complete real AArch64/Linux applications.

The trace-model performance metric is a cycle proxy. Demand cache hits cost 4 cycles, misses cost 100 cycles, and useful prefetch fills cost 8 cycles. The gem5 performance metric is simulated ticks, backed by cache and MSHR statistics. The two models serve different roles: the trace model stress-tests the safety invariant under adversarial events, while gem5 tests whether proof-gated runahead produces a measurable timing effect in an ARM-style memory hierarchy.

7. Results

7.1 Main Synthetic Trace

On the main synthetic trace, naive DMP is fastest but unsafe. COPPER-LINE preserves a large fraction of the speedup while eliminating modeled unsafe prefetches.

Policy	Speedup	Prefetches	Data-at-rest	Cross-domain	Unproven value	Unproven line
disabled	1.000x	0	0	0	0	0
naive	3.628x	4032	2048	1076	2616	2616
copper_value	2.414x	1416	0	0	0	0
copper_line	2.414x	1416	0	0	0	0
copper_stream	1.641x	944	0	0	0	0

The important comparison is between copper_value and copper_line. Both are safe in this trace, but COPPER-LINE does not require a global value table. Its proof state scales naturally with cache lines.

7.2 Adversarial Trace

The adversarial trace tests the exact cases reviewers will ask about.

Policy	Speedup	Prefetches	Data-at-rest	Cross-domain	Unproven value	Unproven line
disabled	1.000x	0	0	0	0	0
naive	1.020x	7	1	2	4	3
copper_value	1.000x	1	0	0	0	0
copper_line	1.000x	1	0	0	0	0
copper_stream	1.000x	1	0	0	0	0

COPPER-LINE allows only the clean-after-proof case and blocks data-at-rest, first-use, stale rewrite, cross-domain, translation-fail, and permission-fail cases.

7.3 Monte Carlo Stability

Across 30 seeds, COPPER-LINE remains stable because the benign pointer structure is unchanged while the secret-like values vary.

Policy	Mean speedup	Std. dev.	Min	Max	Mean data-at-rest	Mean cross-domain	Mean unproven line
disabled	1.000x	0.000	1.000x	1.000x	0.0	0.0	0.0
naive	3.644x	0.026	3.594x	3.708x	2048.0	1014.1	2616.0
copper_value	2.414x	0.000	2.414x	2.414x	0.0	0.0	0.0
copper_line	2.414x	0.000	2.414x	2.414x	0.0	0.0	0.0
copper_stream	1.641x	0.000	1.641x	1.641x	0.0	0.0	0.0

7.4 Rewrite Sensitivity

As the fraction of rewritten pointer fields grows, all safe policies lose prefetch opportunities by design. COPPER-LINE's security behavior does not degrade.

Rewrite fraction	Naive speedup	COPPER-value speedup	COPPER-LINE speedup	Naive unproven line	COPPER-LINE unproven line
0.00	4.080x	2.601x	2.601x	2544.0	0.0
0.01	3.998x	2.568x	2.568x	2556.0	0.0
0.05	3.660x	2.414x	2.414x	2616.0	0.0
0.10	3.333x	2.246x	2.246x	2691.0	0.0
0.25	2.738x	1.858x	1.858x	2916.0	0.0
0.50	2.380x	1.445x	1.445x	3288.0	0.0

7.5 Value-Table Capacity Stress

The earlier value-bound design depends on global table capacity. COPPER-LINE does not.

Value table entries	COPPER-value speedup	COPPER-LINE speedup	COPPER-value prefetches	COPPER-LINE prefetches
0	1.000x	2.414x	0.0	1416.0
64	1.000x	2.414x	0.0	1416.0
128	1.000x	2.414x	0.0	1416.0
256	1.000x	2.414x	0.0	1416.0
512	2.414x	2.414x	1416.0	1416.0
1024	2.414x	2.414x	1416.0	1416.0

This is the most important refinement found during the project. A global value table creates capacity and collision concerns. Line-resident clean proof avoids that limitation.

7.6 Metadata Cost

Assuming 64-byte lines and eight 64-bit words:

Domain bits per line	Metadata bits per 64B line	Data-array overhead
0	8	1.56%
4	12	2.34%
8	16	3.12%
16	24	4.69%

The proof bits are small relative to data storage. The domain color can often be represented compactly because the cache already tracks physical tags, state, and protection context outside the data array.

7.7 RTL and Vivado Results

Vivado xsim passes all directed tests plus a randomized invariant monitor:

```
COPPER gate directed tests completed
COPPER stream gate directed tests completed
COPPER stream table gate directed tests completed
COPPER line provenance directed tests completed
COPPER line provenance random invariant tests completed: trials=2000 allowed=339 blocked=1152
errors=0
COPPER commit-epoch proof bridge directed tests completed
COPPER full authority gate tests completed: directed=12 random=5000 allowed=956 blocked=3731
stale=624 token=123 target=240 terminal=58 perm=183 errors=0
COPPER CLPD gate tests completed: directed=14 random=5000 allowed=4 blocked=5012 no_entry=4864
word_unproven=12 stale_epoch=132 token=2 fault_perm=2 write_clear=1 fill_clear=1
invalidate_clear=1 collision=1 errors=0
```

The randomized monitor is deliberately independent of the RTL implementation. It mirrors the architectural metadata lifecycle in a scoreboard and checks that dmp_seed_allow, dmp_seed_block, and source_proven_clean match the COPPER-LINE invariant across mixed event schedules.

The CEPF bridge test covers clean proof creation, non-memory commits, squashes, missing source tags, architectural exceptions, translation failures, permission failures, and stale source epochs. The stale epoch case is the key backend race: a dependent operation may carry an old source tag, but proof cannot be created if the current source epoch no longer matches the captured epoch.

The full-authority gate adds a combined CEPF/PASB/CTLW/terminal RTL predicate: 12 directed plus 5,000 randomized named-coverage trials pass with 0 mismatches. The CLPD gate adds a retained source-line proof-mask directory: 14 directed tests plus 5,000 randomized scoreboard trials pass with 0 mismatches, including write/fill/invalidate clearing and direct-mapped collision eviction.

Synthesis results:

RTL block	Slice LUTs	Slice registers	BRAM	DSP	WNS at 10 ns	Worst data path
copper_line_provenance_gate	2063 / 20800, 9.92%	1024 / 41600, 2.46%	0	0	+8.122 ns	1.727 ns
copper_commit_epoch_proof_bridge	5 / 20800, 0.02%	0 / 41600, 0.00%	0	0	+3.682 ns	6.318 ns
copper_stream_table_gate	2528 / 20800, 12.15%	2209 / 41600, 5.31%	0	0	+0.232 ns	9.386 ns

The core line-provenance gate is fast because it is an indexed metadata check. The CEPF bridge is small because it only qualifies commit metadata and compares a captured source epoch against the current epoch. The stream-table extension is much closer to the timing limit because it uses direct CAM-like comparisons. The paper should present COPPER-LINE and CEPF as the core proposal and COPPER-STREAM only as an optional research extension.

The earlier Vivado Tcl initialization blocker was environmental, not an RTL failure. A clean TclStore/AppData batch flow now synthesizes the compact CLPD gate at 3,742 LUTs, 2,624 registers, 0 BRAM, and WNS +0.979 ns on xc7a35tcpg236-1; the scalable CLPD SRAM directory and PEB refresh in Section 7.10 give the production-capacity area/timing points. The combined full-authority predicate remains simulation/SVA evidence rather than a production physical implementation.

7.8 Graph-Style Provenance Trace

The graph-style trace is a broader pointer-array stress test than a linked list. It uses 4,096 nodes, 32,768 CSR-like edge slots, five repeated passes, ten seeds, a pointer-shaped adversarial side array, and a 5% edge rewrite after warmup.

Policy	Speedup vs disabled	Demand misses	Prefetches	Data-at-rest PF	Stale/unproven PF	Epoch/value blocks
disabled	1.000x	162653.1	0.0	0.0	0.0	0.0
naive	7.991x	0.0	174080.0	10240.0	0.0	0.0
source-only provenance	3.361x	32768.0	131072.0	0.0	1638.0	0.0
COPPER epoch/value	3.276x	34204.1	129434.0	0.0	0.0	1638.0

Naive DMP is fastest, but it prefetches all adversarial pointer-shaped data-at-rest values. Source-only provenance looks attractive but fails after edge rewrites: a previously proven slot authorizes 1,638 changed values on average. COPPER epoch/value provenance blocks those stale rewritten-edge cases while preserving most of the repeated graph-pass speedup.

We then replace the synthetic graph generator with public GAPBS serialized graph files. The parser validates the .sg header, CSR offsets, and 32-bit neighbor array, then replays edge-scan and BFS-replay streams over five generated Kronecker graphs: g10, g11, g12, g13, and g14. This is still a trace model, not official full-system GAPBS, but it ties the DMP/provenance question to GAPBS topology rather than an invented graph.

Policy	Speedup	Demand misses	Prefetches	Data-at-rest PF	Unproven edge PF	Stale slot PF	Epoch/value blocks
disabled	1.000x	80,124.6	0.0	0.0	0.0	0.0	0.0
naive	3.444x	68.2	298,711.0	16,384.0	148,063.5	900.8	0.0
source-only provenance	1.783x	51,257.5	135,164.2	0.0	900.8	900.8	0.0
COPPER epoch/value	1.770x	140,166.9	126,931.3	0.0	0.0	0.0	851.3
COPPER-CLPD	1.896x	56,266.9	287,642.3	0.0	0.0	0.0	29,824.8

The GAPBS-backed trace confirms the safety separation on real generated graph topology. Naive DMP is faster in the proxy but issues data-at-rest and unproven-edge prefetches. Source-only proof blocks data-at-rest but still permits stale rewritten slots. COPPER epoch/value blocks both, and COPPER-CLPD keeps the same zero-unsafe counters while being more conservative after source-line writes.

The capacity sweep exposes and fixes a real design constraint. With g12 edge-scan reuse, the edge-exact ledger needs 131,072 entries to recover 2.369x speedup; smaller edge ledgers correctly block but do not help. CLPD reaches 2.115x with 8,192 source-line entries because one retained line proof covers up to sixteen 32-bit edge slots. The tradeoff is conservative invalidation: writes to any slot in a source line block all words in that line until demand recreates proof.

Policy	Proof entries	Speedup	Useful PF hits	Epoch/value blocks
edge-exact	65,536	1.000x	0	0
edge-exact	131,072	2.369x	68,756	1,935
CLPD	4,096	1.000x	0	290,220
CLPD	8,192	2.115x	62,713	26,624

The storage model makes this tradeoff concrete. With 64-byte source lines, 4-byte edge slots, a 16-bit token, an 8-bit epoch, and a 64-bit value field for edge-exact entries, CLPD full coverage is about 30.86-32.00x smaller than an edge-exact retained-value ledger across the GAPBS-generated graphs. On g12, the edge-exact full-cover proxy is 1252.18 KiB versus 39.87 KiB for CLPD. At the measured g12 capacity points, 8192 CLPD entries cost 54.00 KiB and recover 2.115x speedup, while the edge-exact ledger needs 131,072 entries, 1696.00 KiB, to recover 2.369x. These are storage proxies, not physical SRAM/CAM reports, but they explain why CLPD is a hardware-relevant refinement instead of merely a larger proof table.

The expanded kernel sensitivity sweep then asks whether this safety behavior is a single-trace artifact. It reuses the three largest GAPBS-generated topologies and runs PageRank-pull, SSSP-relaxation, connected-components label propagation, and triangle-counting oriented-edge traces across proof entries, cache sizes, lookahead distances, and seeds. Across 4,320 policy/configuration runs, both COPPER variants keep the unsafe modeled prefetch count at zero.

Policy	Runs	Mean speedup	Median speedup	Unsafe total	Worst unsafe/run	Mean no-proof blocks
disabled	864	1.000x	1.000x	0	0	0.0
naive	864	5.615x	5.881x	81,605,320	110,568	0.0
source-only	864	1.309x	1.000x	284,488	1,404	94,121.3
COPPER-epoch	864	1.295x	1.000x	0	0	94,286.0
COPPER-CLPD	864	1.649x	1.885x	0	0	68,794.7

This sweep is a model-level result, but it is useful reviewer evidence. Naive DMP shows the upper performance bound while also dereferencing huge numbers of unsafe pointer-shaped values. Source-only proof remains unsafe after rewrites. Edge-exact COPPER is safe but loses benefit under small ledgers, while CLPD recovers median speedup by compressing clean source-line proof without relaxing the value/epoch safety invariant.

7.9 Free-Tool Baselines

ChampSim was built under a local MSYS2 toolchain after two portability fixes: relative include paths for response files and an explicit iterator-distance type conversion for MinGW. We generated three deterministic ChampSim traces: sequential scan, pointer chase, and adversarial-shaped dependent loads. Stock ChampSim traces do not encode committed pointer-source words or security domains, so these runs evaluate ordinary cache/prefetch behavior, not the COPPER invariant.

Prefetcher	Trace	IPC	L1D miss rate	L1D misses	Issued prefetches	Useful prefetches
no	sequential_scan	0.86120	0.00673	135	0	0
next_line	sequential_scan	0.86083	0.00943	253	6769	0
ip_stride	sequential_scan	0.86120	0.00673	135	0	0
no	pointer_chase	0.81626	0.00967	194	0	0
next_line	pointer_chase	0.82566	0.01026	262	5506	7
ip_stride	pointer_chase	0.81626	0.00967	194	0	0
no	adversarial_shape	0.93533	0.00713	119	0	0
next_line	adversarial_shape	0.93625	0.00978	202	3960	0
ip_stride	adversarial_shape	0.93687	0.00775	130	117	0

The baseline result is not that COPPER is faster than these prefetchers. The result is that ordinary address-stream prefetchers are not a substitute for a DMP authority rule. Next-line issues many low-usefulness prefetches on pointer-shaped traces, while IP-stride mostly does not activate.

GAPBS was also built and verified locally. The largest generated-graph runs used scale 20, corresponding to 1,048,576 nodes and 15,699,691 undirected edges. BFS, connected components, PageRank, and SSSP all passed verification. These results demonstrate that the workstation can run graph kernels relevant to pointer-heavy behavior, but they are not yet connected to COPPER because the current GAPBS run is native execution rather than trace-level DMP/provenance simulation.

7.10 ARM gem5 COPPER Prototype

The gem5 prototype is the strongest new evidence from this pass. It is still syscall-emulation and synthetic, but unlike the trace model it exercises an ARM CPU model, timing caches, MSHRs, an L2, memory timing, and gem5 prefetcher issue paths.

Headline full-list results:

Workload	Prefetcher	Speedup vs none	Ticks	Demand MSHR misses	PF MSHR misses	PF issued	Carried hits	Translated PF
Sequential 8192-node list	none	0.00%	3514397000	33026	0	0	0	0
Sequential 8192-node list	recursive COPPER	6.76%	3291910000	8451	25260	25260	25259	394
Sequential 8192-node list	stride	98.93%	1766614000	522	32504	32504	0	0
Page-permuted 8192-node list	none	0.00%	3514397000	33026	0	0	0	0
Page-permuted 8192-node list	recursive COPPER	6.76%	3291910000	8451	25266	25266	25265	394
Page-permuted 8192-node list	stride	0.69%	3490348000	32776	250	250	0	0
Fully random 8192-node list	none	0.00%	4119013000	33026	0	0	0	0
Fully random 8192-node list	same-page COPPER	0.06%	4116483000	32828	198	198	0	0
Fully random 8192-node list	one-hop COPPER + CPTQ	2.74%	4008993000	20738	12288	12288	0	12180
Fully random 8192-node list	recursive COPPER	5.64%	3899053000	8451	25166	25166	25165	24959
Fully random 8192-node list	stride	0.59%	4095023000	32776	250	250	0	0
Medium 256-node page-permuted list	none	0.00%	52705000	1030	0	0	0	0
Medium 256-node page-permuted list	recursive COPPER	15.06%	45805000	263	818	818	817	12
Medium 256-node page-permuted list	stride	0.17%	52616000	1029	2	2	0	0

Across page-permuted seeds 1-5, recursive COPPER gives 6.76-6.78% speedup while stride gives 0.69%. Across random seeds 1-3, recursive COPPER gives 5.59-5.66% speedup while stride gives 0.59%. The result is not a raw D-cache miss-count reduction: most runs keep the same 33,026 D-cache misses. The performance effect comes from moving misses out of the demand-visible path. For random seed 1, demand MSHR misses fall from 33,026 to 8,451 while COPPER creates 25,166 prefetch-origin MSHR misses.

This is exactly the behavior expected from the RCP/CPTQ mechanism. Same-page COPPER barely helps a random list. CPTQ recovers cross-page one-hop issue. RCP then allows already-proven prefetched lines to carry enough context to seed the next safe prefetch, yielding the strongest random and page-permuted results. The sequential case remains a deliberate non-headline: stride prefetching nearly doubles speed because the layout is trivially regular.

We also generated tiny AArch64 Linux ELF pointer benchmarks directly, avoiding a missing cross-compiler. These use 64-bit AArch64 instructions with 32-bit pointer fields so they exercise the same COPPER source-word path. On full 8192-node AArch64 runs, recursive COPPER improves page-permuted lists by 6.77% versus 0.67% for stride, and random lists by 5.61% versus 0.57% for stride. Demand MSHR misses again fall from about 33,024 to 8,449.

CPU-model sensitivity is positive but more modest on deeper cores:

AArch64 model/workload	Recursive COPPER	Stride
TimingSimple page-permute	6.77%	0.67%
TimingSimple random	5.61%	0.57%
Minor page-permute	2.79%	0.59%
Minor random	2.64%	0.50%
O3 page-permute	2.77%	0.13%

AArch64 model/workload	Recursive COPPER	Stride
O3 random	2.68%	0.11%

The O3 traffic proxy is also important. For page-permuted AArch64, recursive COPPER increases DRAM read bytes from 2,113,920 to 2,124,416 (+0.50%) and L2 misses from 33,030 to 33,194 while giving 2.77% speedup. For random AArch64, it increases DRAM read bytes from 2,113,920 to 2,123,968 (+0.48%) and L2 misses from 33,030 to 33,187 while giving 2.68% speedup. This does not prove production energy efficiency, but it argues that the measured O3 gain is not purchased with a large bandwidth explosion in these pointer-chain runs.

Finally, the full-system ARM64 path now has both an environment probe and an integrated native workload ROI. With the public ARM Ubuntu 24.04 image and Linux 6.8.12, the probe mounted /dev/vda2, loaded gem5_bridge, executed the readfile script, printed COPPER_FS_PROBE_START and COPPER_FS_PROBE_DONE, reported aarch64, and exited after 470,650,241 simulated instructions. The native-workload run then skipped guest Python startup overhead, decoded and executed a static AArch64 ELF, switched from atomic boot to timing CPU at m5 workbegin, reset statistics immediately before the binary, dumped an ROI after the workload completed, and attached the selected L1D prefetcher in the full-system cache hierarchy.

Full-system ARM64 metric	Result
Guest ISA reported by Linux	aarch64
Linux kernel	6.8.12
Boot/readfile probe instructions	470,650,241
Native ELF ROI instructions	2,712,437
Native ELF ROI ticks	970,923,105
Native ELF ROI core-0 IPC	0.731638
Native ELF ROI core-1 IPC	0.197506
Native workload markers	COPPER_FS_NATIVE_A64_START to COPPER_FS_NATIVE_A64_DONE rc=0

The native AArch64 workload prints the same policy contrast that motivates CEPF: naive DMP obtains a 5.675x speedup proxy but issues 128 data-at-rest prefetches; source-only provenance gives 2.702x but still permits 6 stale/unproven rewritten-source prefetches; COPPER epoch/value gives 2.655x while blocking both data-at-rest and stale rewritten-source activation. The integrated timing run then exposes the hardware-prefetcher side:

Full-system timing-ROI prefetcher	ROI ticks	vs none	L1D misses	Prefetches issued	Useful prefetches	Pointer-like candidates	Learned proofs	Allowed candidates	Blocked candidates	Translation faults
none	3,571,493,265	0.000%	38,899	0	0	0	0	0	0	0
stride	3,382,963,983	-5.279%	32,022	15,443	7,125	0	0	0	0	0
naive pointer-shaped DMP	3,572,333,757	+0.024%	38,900	40	0	70	32	70	0	30
COPPER before PASB	3,571,493,265	0.000%	38,899	0	0	102	69	5	97	5
COPPER-PASB	3,571,493,265	0.000%	38,899	0	0	102	63	0	102	0

This first table is an important negative and positive result. Stride is the right tool for the simple stream-like parts of this tiny native workload and gives 5.279% speedup. A naive pointer-shaped DMP does not help: it issues 40 prefetches, none useful, and produces 30 faulted recursive translation attempts. COPPER before PASB is safer but still lets 5 recursive candidates reach translation and fault. COPPER-PASB blocks all 102 pointer-shaped candidates that lack address-space-bound committed authority and eliminates those full-system translation faults. Real AArch64/Linux execution therefore forced a stronger proof-key invariant.

The larger full-system runs then exposed the second refinement, CTLW-terminal. PASB-only COPPER gave small positive timing movement on the larger page-permuted pointer ROI but still recorded 8 recursive translation faults. A naive page-level committed-translation witness removed those faults but over-opened recursion, issuing 85,377 prefetches and slowing the same ROI by 3.264%. Exact line witnesses plus terminal witness-derived fills produced the balanced result below.

Shape	Run	ROI ticks	vs none	L1D misses	L2 data misses	Prefetches issued	Useful prefetches	Pointer-like candidates	Learned proofs	Allowed	Blocked	Translation faults	CTLW hits	Terminal stops
page-permute	none	5,174,659,494	0.000%	76,239	54,733	0	0	0	0	0	0	0	0	0
page-permute	stride	4,872,406,383	-5.841%	66,517	43,702	19,291	10,928	0	0	0	0	0	0	0
page-permute	PASB-only	5,152,852,656	-0.421%	76,244	30,153	25,053	0	40,318	8,220	25,061	15,257	8	0	0
page-permute	CTLW-terminal	5,147,204,976	-0.531%	76,236	30,539	24,229	0	39,487	8,219	24,229	15,258	0	392	392
random	none	5,321,033,973	0.000%	75,792	54,873	0	0	0	0	0	0	0	0	0
random	stride	5,058,705,897	-4.930%	66,004	43,679	19,270	10,793	0	0	0	0	0	0	0
random	naive DMP + CTLW	5,307,171,516	-0.261%	75,800	42,579	12,565	0	27,433	8,148	27,433	0	0	12,426	0
random	COPPER CTLW-terminal	5,306,601,753	-0.271%	75,795	42,537	12,341	0	27,600	8,204	12,341	15,259	0	12,236	12,238

These larger runs are still generated pointer workloads, not SPEC or GAP inside Linux. They nevertheless close the earlier full-system hole: COPPER now has a measurable AArch64/Linux cache-path result with no recursive translation faults, modest positive timing movement, and visible source-authority behavior. On the random shape, naive DMP plus CTLW allows all 27,433 pointer-shaped candidates, while COPPER CTLW-terminal allows 12,341 and blocks 15,259 unproven candidates with nearly the same timing. CTLW is therefore not the COPPER security mechanism by itself; it is the translation witness that makes recursive COPPER safe to carry through full-system execution.

To reduce the "linked-list-only" concern, we also generated a full-system AArch64 graph-gather binary. It uses stable CSR-like edge slots, 4096 target nodes, degree 4, four passes, random targets, a short compute gap between edge-pointer load and target-node demand load, and a 4096-entry pointer-shaped fake side array that is read but never dereferenced by demand code.

Run	ROI ticks	vs none	L1D misses	L2 data misses	Accesses	Prefetches issued	Useful prefetches	Pointer-like candidates	Learned proofs	Allowed	Blocked	Translation faults	CTLW hits	Terminal stops
none	5,272,379,010	0.000%	91,676	29,610	795,524	0	0	0	0	0	0	0	0	0
stride	4,831,167,330	-8.368%	80,745	19,555	796,314	15,636	11,337	0	0	0	0	0	0	0
naive DMP + CTLW	5,251,114,629	-0.403%	88,568	29,139	795,524	8,110	3,108	11,479	5,030	11,479	0	0	4,091	0
COPPER CTLW-terminal	5,253,046,029	-0.367%	89,234	29,162	795,524	49,891	2,438	58,551	5,030	49,891	8,660	0	3,075	10,834

This graph-gather result is not a replacement for GAPBS or SPEC, but it is a stronger application-shaped control than a pure pointer chain. Stride wins because the edge array itself is sequential and prefetchable. Naive DMP is slightly faster than COPPER, but it permits every pointer-shaped candidate. COPPER is within 0.036 percentage points of naive timing, blocks 8,660 unproven candidates, and keeps recursive translation faults at zero. The result supports the paper's narrower claim: COPPER can preserve a measurable subset of DMP benefit while enforcing source authority, not that COPPER beats a conventional stride prefetcher on every mixed graph loop.

The final local run adds a compiler-authored C workload. LLVM/clang 22.1.7 and LLD compile a freestanding AArch64 binary with no libc or dynamic loader. The source implements graph gather, hash-table probing, binary-tree lookup, and a fake pointer-shaped side array. All full-system runs print the same checksum, 0x5bf8bf1b.

Run	ROI ticks	vs none	Instructions	L1D misses	L2 data misses	Accesses	Prefetches issued	Useful prefetches	Pointer-like candidates	Learned proofs	Allowed	Blocked	Translation faults	CTLW hits	Terminal stops
none	4,600,705,023	0.000%	4,871,243	84,379	15,527	1,099,947	0	0	0	0	0	0	0	0	0
stride	4,459,479,390	-3.070%	4,878,761	76,877	9,329	1,099,680	17,118	9,234	0	0	0	0	0	0	0
naive DMP + CTLW	4,599,334,062	-0.030%	4,871,243	84,329	15,495	1,099,947	1,266	66	1,583	305	1,583	0	0	1,263	0
COPPER CTLW-terminal	4,605,001,389	+0.093%	4,871,243	84,332	15,523	1,099,947	904	62	1,583	305	904	679	0	902	1,611

This C-suite result is an honest neutral/negative performance point. COPPER is slightly slower than no prefetch, and stride is clearly best because the compiled workload includes sequential initialization, array scans, and table walks. The useful contribution is methodological and security-oriented: the evaluation now includes compiler-authored AArch64 code under full-system Linux, where COPPER blocks unproven pointer-shaped candidates and avoids recursive translation faults.

We then added a second compiler-authored full-system workload shaped after GAPBS kernels, while keeping it small enough for local full-system gem5 iteration. It is not official GAPBS. The source implements BFS, SSSP-like relaxation, PageRank-style gather, and connected-components-style propagation over pointer-valued adjacency arrays, plus a fake pointer-shaped side array that is read but never dereferenced. All runs print checksum 0xf1dd4e4d.

Run	ROI ticks	vs none	Instructions	L1D misses	L2 data misses	Accesses	Prefetches issued	Useful prefetches	Pointer-like candidates	Learned proofs	Allowed	Blocked	CTLW hits	CTLW misses	Terminal stops	Translation unavailable	Translation faults
none	3,770,630,928	0.000%	4,033,667	47,896	11,434	751,422	0	0	0	0	0	0	0	0	0	0	0
stride	3,660,924,078	-2.910%	4,036,927	38,929	6,438	749,195	15,896	10,313	0	0	0	0	0	0	0	0	0
naive DMP + CTLW	3,770,703,522	+0.002%	4,033,667	47,872	11,434	751,422	1,835	101	2,243	417	2,243	0	1,824	408	0	408	0
COPPER CTLW-terminal	3,770,630,928	0.000%	4,033,667	47,877	11,434	751,422	1,288	56	2,240	417	1,288	952	1,280	0	7,729	0	0

The GAPBS-inspired mini-suite strengthens the real-workload direction but also makes the performance boundary clearer. COPPER is not faster than no prefetch on this reduced graph-kernel run; stride wins by 2.910% because edge arrays are still sequential. The COPPER result is a control result: relative to naive DMP+CTLW, COPPER blocks 952 unproven candidates, removes all CTLW misses and unavailable recursive translations, keeps recursive translation faults at zero, and enforces terminal witness-derived fills.

We then reran the same GAPBS-inspired C source at the larger compile-time size used by the local binary aarch64_gapbs_mini_suite_fs: 1024 vertices, degree 8, 3 passes, and 2048 fake pointer-shaped words. This corrected timing-mode run uses the full-system m5 workbegin switch, resets stats immediately before the native AArch64 binary, and dumps the first ROI section immediately after return. All four runs print checksum 0x0ba8df31.

Run	ROI ticks	vs none	Instructions	L1D misses	Prefetches issued	Useful prefetches	Pointer-like candidates	Learned proofs	Allowed	Blocked	CTLW hits	CTLW misses	Terminal stops	Translation unavailable	Translation faults
none	7,449,479,397	0.000%	13,098,729	149,169	0	0	0	0	0	0	0	0	0	0	0
stride	7,373,493,459	-1.020%	13,256,604	129,837	36,808	22,955	0	0	0	0	0	0	0	0	0
naive DMP + CTLW	7,431,646,914	-0.239%	13,098,702	145,214	6,289	4,061	6,295	1,090	6,295	0	6,271	6	0	6	0
COPPER CTLW-terminal	7,434,010,881	-0.208%	13,098,729	145,813	4,953	3,414	6,293	1,089	4,953	1,340	4,937	0	50,737	0	0

The larger run is still not official GAPBS, but it gives a less tiny full-system graph-kernel control. COPPER remains a near-neutral performance point, blocks 1,340 unproven pointer-shaped candidates, removes the six CTLW misses and six unavailable recursive translations seen by naive DMP+CTLW, and keeps recursive translation faults at zero. The result is useful because it scales the full-system safety/control behavior without changing the honest performance conclusion: conventional stride is still best when the graph kernel spends much of its time scanning sequential edge arrays.

The bounded invariant checker gives a compact formal-style sanity check for the refinements discovered during full-system testing.

Variant	Result	States explored	Counterexample
COPPER-PASB-CTLW-terminal	PASS	42	none within depth 10
no PASB	FAIL as expected	12	committed proof, then context switch, then unsafe issue
no CTLW	FAIL as expected	13	committed proof, cross-page target, no line witness
no terminal rule	FAIL as expected	24	CTLW-derived fill recursively chased without demand validation

The checker is deliberately small: it does not prove the whole gem5 implementation, but it shows that the three named rules correspond directly to the three bug classes found in testing.

A richer bounded checker then adds source values, source epochs, proof soundness, CEPF-like in-flight backend source tags, address-space tokens, exact target-line witnesses, witness invalidation, and terminal fills. The full authority rule passes 11,419 reachable states to depth 12. Weakened CEPF, source-invalidation, PASB, CTLW, page-witness, remap-witness, and terminal-fill variants fail with short counterexamples. This still is not industrial formal signoff, but it raises the proof standard above one-off directed tests.

An assertion-focused full-authority SVA harness then checks the RTL predicate directly under Vivado XSIM. It passes 12 directed cases plus 10,000 randomized samples with nonzero coverage for no-source, unsound proof, stale value, stale epoch, PASB token mismatch, terminal-source, missing witness, wrong-line witness, stale witness, permission failure, same-page allow, and cross-page allow classes. The harness asserts that any allowed seed requires exact committed source proof, token match, non-terminal source status, target authority, and permission success.

An end-to-end CEPF-to-line SVA harness then connects the backend proof filter to the line-resident proof gate. It passes 12 directed cases plus 10,000 randomized samples with valid_commit=2257, proof_to_allow=769, unproven_block=7658, stale_epoch_block=151, fault_perm_block=1321, write_clear=1, fill_clear=1, invalidate_clear=1, and errors=0. This closes the previous local evidence gap between proof creation and DMP gate use: valid CEPF proof can authorize a matching line-gate query, while stale, faulted, unproven, squashed, destructive, or permission-invalid paths cannot.

A CTLW witness-directory RTL block then checks the target-line witness object itself. It passes 10 directed cases plus 10,000 randomized XSIM samples with exact_hit=1484, miss=6712, token_mismatch=124, line_mismatch=5162, remap_clear=1, tlbi_token_clear=112, tlbi_all_clear=49, collision=3354, and errors=0. This moves stale remap/TLBI witness handling from a model-only argument to a hardware-shaped artifact.

The CTLW-to-full-authority integration harness then wires the witness directory into the final CEPF/PASB/CTLW/terminal predicate. It passes 12 directed cases plus 10,000 randomized XSIM samples with exact_cross_allow=3, no_witness_block=7102, token_mismatch_block=28, line_mismatch_block=6200, stale_after_remap_block=1, stale_after_tlbi_block=1, terminal_block=274, permission_block=85, stale_source_block=260, collision=2659, and errors=0. This closes the local RTL gap between the CTLW witness object and the DMP issue gate that consumes it.

The combined CLPD-CTLW authority harness wires source-proof compression and target-line witnessing into the same final gate. It passes 18 directed cases plus 10,000 randomized XSIM samples with joint_cross_allow=180, no_source_block=8468, word_unproven_block=181, stale_epoch_block=374, target_no_witness_block=1239, target_line_alias_block=1183, write_clear_block=1, fill_clear_block=1, invalidate_clear_block=1, remap_block=1, tlbi_block=1, terminal_block=54, permission_block=12, clpd_collision=14, ctlw_collision=1376, and errors=0. This closes the local RTL gap between CLPD compression and CTLW authority consumption.

A SARI RTL harness then checks the SoC/coherence revocation boundary. It passes 8 directed cases plus 10,000 randomized XSIM cycles with dma=1, chi=1, io=1, triple_burst=1, hold=6321, remap=1, tlbi_token=1, tlbi_all=1, ready_low=4, overflow=4, final_queue=0, and errors=0. SARI is not a complete CHI implementation; it is a testable boundary contract that maps DMA/CHI/coherent-I/O writes to queued CLPD source revocations, maps remap/TLBI events to CTLW, and immediately holds DMP issue while revocation is pending.

CS-SARI then removes SARI's global-stall limitation without relaxing the safety rule. Its wired authority harness passes 12 directed plus 10,000 randomized XSIM samples with conflict_hold=1245, matching_source_hold=1, queued_source_hold=1, matching_remap_hold=1, matching_tlbi_hold=1, tlbi_all_hold=1, avoided_global_hold=1007, avoided_global_allow=1007, and errors=0. The GAPBS-topology revocation proxy reports three scenarios over 6,335,916 raw candidates each: hold reduction is 96.53% for low-conflict revocation noise, 58.08% for hot shared-buffer revocations, and 84.07% under TLBI churn, with 269,879 aggregate authorized candidates recovered versus global hold and zero modeled unsafe issues for CS-SARI. A bounded composition checker then wires the CS-SARI hold predicate to live CLPD source proof, exact CTLW target witnesses, token state, remap/TLBI events, and overflow fallback. The full composed rule passes all 7,555 reachable states; weakened variants that omit incoming-source, queued-source, remap, token-TLBI, or global-TLBI holds fail with stale-authority counterexamples. A 20-configuration sensitivity sweep across queue depths 1, 2, 4, 8, and 16 plus low-conflict, balanced, hot-source, and target-churn profiles reports zero CS-SARI unsafe modeled issues, 1,649,883 no-hold unsafe modeled issues, 447,509 avoided global holds, and a 72.06% median scoped-hold reduction.

A CLPD-specific state-space checker then compares the compressed directory against ground-truth committed source proofs. The model uses three source lines, two direct-mapped directory entries, two words per line, two address-space tokens, and one-bit source-line epochs. Full CLPD passes 24,354 reachable states to depth 8. Weakened variants without tag checks, token checks, epoch checks, per-word proof masks, write clearing, fill clearing, or invalidate clearing all fail with short counterexamples. This directly checks that CLPD is a safe representation of COPPER authority, not just a capacity optimization.

A generated security coverage matrix then audits the evidence chain across ten modeled unsafe classes: pointer-shaped data at rest, first-use source words, cross-domain/token mismatch, stale source proof, CLPD alias and whole-line overreach, external DMA/coherence update races, missing cross-page target witnesses, stale target witnesses, recursive terminal-fill amplification, and permission/translation failure. The generator performs an evidence string audit by checking that each row's claimed local evidence is present before marking the row covered; the current output is PASS. The newest matrix also cites the CS-SARI composition checker and sensitivity sweep for source revocation, queued revocation, DMA/coherence race, target remap, and TLBI cases. This is not a complete security proof, but it makes the paper's modeled security coverage and residual risks source-backed rather than narrative-only.

7.10 ROI-Bracketed Heap, PEB, and Scalable CLPD Refresh

The strongest current full-system performance evidence is a ROI-bracketed AArch64 heap-pointer stress workload, not the broad GAPBS suite. The workload allocates 32,768 heap nodes, traverses real pointers for 16 passes, scans 32,768 fake pointer-shaped data words for four passes, performs pointer rewrites, and prints a checksum. It calls m5_reset_stats after construction and m5_dump_stats before exit, so the first statistics window measures the ROI rather than allocation/setup. Across three heap-layout seeds, CLPD-64K improves ROI ticks on all seeds with mean -2.866% versus no-prefetch; naive DMP slows all three seeds with mean +15.214% and 1,568,208 aggregate CTLW misses.

The fake-only control isolates the data-at-rest failure mode by setting --passes=0 --rewrite=0. Naive DMP is +396.087% and issues 28,685 prefetches from non-dereferenced fake data. CLPD-64K without a boundary blocks 99.99542% of pointer-shaped observations but still permits 6 warm-state candidates. PEB fixes that precisely: CLPD-64K+PEB blocks 131,066/131,066 fake observations, issues zero prefetches, drops 76,560 pre-boundary authority entries, and is +0.033% versus no-prefetch. The same PEB mode still improves the three real heap seeds with mean -2.905%, zero CTLW misses, zero translation faults, and matching checksums.

Official GAPBS now runs as AArch64 Linux binaries in full-system gem5. The six-kernel g10 suite is not pointer-heavy, because GAPBS stores most graph edges as integer vertex IDs, but it is useful external-validity evidence. CLPD-64K+PEB runs BFS/CC/PR/SSSP/BC/TC with all rc=0, +0.015% aggregate timing, zero translation faults, zero proof evictions, CTLW misses reduced from naive's 47,176 to 482, and 340,128 pre-boundary authority entries dropped.

The hardware story also improved. A scalable true-dual-port SRAM-style CLPD directory passes 18 directed plus 4,000 randomized XSIM tests with hazard-closed lookup. The full 64K-entry CLPD configuration used in gem5 synthesizes on xc7a200tfbg676-2 with 629 LUTs, 156 FFs, 260 BRAM tiles, WNS 3.274 ns at 10 ns, and routes out-of-context with 636 LUTs, 170 FFs, 260 BRAM tiles, zero route errors, and post-route WNS 0.362 ns. The PEB epoch/token block is smaller: 346 LUTs, 147 FFs, no BRAM/DSP, WNS 3.782 ns at 10 ns. These are block-level FPGA datapoints, not full ARM/SoC or ASIC signoff.

8. Prior Art

Augury identified DMP data-at-rest leakage on Apple A14/M1-class processors and described an Array-of-Pointers DMP that dereferences pointer arrays. GoFetch demonstrated practical key extraction attacks against constant-time cryptographic implementations using DMP behavior. These works motivate COPPER-LINE but do not propose a committed-provenance hardware gate.

SplittingSecrets is the closest DMP-specific defense found in public work. It uses compiler transformations to prevent secrets from being stored in address-looking form, including support for AArch64 backend-generated memory operations. COPPER-LINE is complementary: it changes hardware authority for DMP dereference rather than transforming selected software.

PreFence is a scheduling-aware defense that disables prefetchers during sensitive regions. It is practical but coarse. COPPER-LINE aims for continuous fine-grained operation.

Pointer-chase prefetchers, indirect prefetchers, ICP, and DX100 show that irregular memory prefetching is a crowded performance field. COPPER-LINE should not be claimed as a new indirect prefetcher. Its claim is security authority for DMP dereference.

SafeSpec and related speculative-side-effect defenses use commit as a boundary, but they target speculative execution leakage. COPPER-LINE targets non-speculative DMP action on data-at-rest or unproven memory contents.

Hardware taint tracking, CHERI, Morello, PICASSO, and ARM MTE are important provenance/tagging relatives. They differ in goal and scope. COPPER-LINE does not enforce architectural memory safety or broad information-flow policy. It supplies a narrow positive permission bit for one microarchitectural consumer: the DMP.

Speculative address-translation work such as Revelator attacks a different bottleneck: predicting or accelerating virtual-to-physical translation for ordinary memory requests. CTLW is narrower and more conservative. It does not predict a physical address from a virtual address; it reuses an exact demand-observed target-line translation only after source provenance and PASB authority already hold, and it prevents witness-derived fills from recursively amplifying until demand-validated. To the best of public knowledge, public translation-prefetch work does not use an exact committed target-line witness as an authority gate for recursive DMP.

Epoch and self-invalidation mechanisms are also prior art. ASIDs/PCIDs, TLB shootdowns, self-invalidating TLB entries, and eventually consistent TLB work all show that stale microarchitectural entries can be managed with tags, epochs, expiration, barriers, or deferred invalidation. Intel's public DDP guidance also describes isolation, barriers, recursive-dereference limits, and disable controls for deployed data-dependent prefetch behavior. COPPER should not claim that per-domain epoch invalidation is new. The PEB claim is narrower: the DMP proof directory's authority is salted by the current provenance epoch, so pre-boundary committed-source proofs cannot authorize post-boundary DMP issue, and the mechanism is evaluated with fake-data controls, full-system AArch64 workloads, and RTL synthesis.

9. Limitations

This project is not silicon signoff. The current evaluation uses synthetic and adversarial COPPER traces, graph-style provenance traces including GAPBS-generated topology replay and an expanded GAPBS-style kernel sensitivity sweep, stock ChampSim synthetic traces, native GAPBS generated-graph runs, ARM/AArch64 gem5 syscall-emulation pointer binaries, an ARM64 full-system Linux boot/readfile probe, native AArch64 full-system timing ROIs with COPPER attached in the L1D prefetcher path, ROI-bracketed heap-pointer/fake-pointer full-system controls, official AArch64 GAPBS full-system runs, compiler-authored freestanding AArch64 C kernels including small and larger GAPBS-inspired mini-suites, public SQLite/Lua/Duktape/yyjson engine runs, repeated medium/stress public-engine layout seeds, gem5-counter traffic/pollution proxy analysis, bounded authority-state checkers, a bounded OoO-LSQ proof-contract checker, a bounded TLB/coherence authority-contract checker, a synthesizable TLB/coherence authority filter, and a CLPD-specific bounded checker. It does not yet include COPPER speedup under gem5 full-system AArch64 Linux execution of SPEC, browser, production database, or cryptographic workloads. Official GAPBS now runs, but its graph representation is mostly integer vertex IDs and is therefore an external-validity/control suite rather than the main pointer-prefetch speedup evidence. The trace performance proxies capture miss-latency behavior and proof/epoch safety, but not MSHR pressure, memory-controller scheduling, cache pollution, wrong-path execution, or real replacement effects. The GAPBS-backed trace uses real generated graph topology but remains a provenance-aware trace replay. The gem5 SE runs do capture MSHR behavior and now include AArch64 TimingSimple, Minor, and O3 sensitivity. The full-system larger CTLW/CLPD/PEB runs capture cache-path behavior under Linux and show small positive timing movement on pointer-heavy controlled workloads, but many workloads are still generated, freestanding static binaries, or bounded public-engine workloads rather than complete production applications. The energy/pollution scorecard is an explicitly weighted proxy over gem5 counters, not calibrated power, DRAMPower, McPAT, or silicon energy.

The design assumes that the implementation can identify committed pointer-source words. The CEPF bridge gives a concrete proof-path guard and cost point, and the new OoO-LSQ proof-contract checker makes the retirement-side obligation executable: proof creation requires dependent-memory retirement, an older retired source load, live epoch/value-matched source tags, no backend flush, and successful target translation/permission. This is still not a production ARM LSQ. A production core would need a careful load-store-queue or dependency-tracking implementation measured for area, timing, replay, exception, and verification complexity in a real backend. The current gem5 implementation models the ledger behavior at the prefetcher/cache boundary; it is not a cycle-accurate backend provenance datapath.

The domain model is now less abstract than before, but still not final. PASB uses a TTBR/CONTEXTIDR-derived token in the gem5 AArch64 prototype. CTLW uses demand-observed virtual and physical cache-line witnesses. A real AArch64-style system would need a precise mapping to ASID, VMID, privilege, translation regime, TrustZone/realm context, PASID, or implementation-specific policy color, plus invalidation on any event that changes that token's authority or remaps a witnessed line.

PEB shows one plausible boundary implementation, but production use must decide which OS, VM, exception-level, ASID/VMID, TLBI, and security-domain transitions increment or clear the provenance epoch. Epoch wrap is explicitly blocked in the RTL until an external proof purge completes; a production design would need to size epochs and verify the purge protocol.

DMA, I/O coherence, and accelerator writes must clear provenance. This is mandatory for AMBA/CHI-connected SoCs. The TLB/coherence authority-contract checker now models these as same-cycle and queued invalidation hazards, along with target remap, token TLBI, global TLBI, exact target-line witnesses, and permission downgrade. The matching RTL filter implements the issue-side hold and exact-witness/permission gate and gives a small cost/timing point. These artifacts are still contract evidence, not a full CHI/ACE/AXI decoder or end-to-end proof.

The generated coverage matrix audits ten modeled unsafe classes against local evidence. The CS-SARI composition checker directly tests the composed source-proof, target-witness, and revocation-hold rule, and the TLB/coherence contract checker separately tests stale witness behavior under source revocation, remap, TLBI, queued invalidation, and permission churn. These artifacts are not a replacement for an end-to-end security proof over a production memory hierarchy.

Finally, industry may already use unpublished DMP safety rules. The correct novelty claim is therefore "to the best of public knowledge," based on public literature and patent/open-source search signals.

10. Reviewer Risk Assessment

The strongest likely rejection is: "This is just taint tracking plus a prefetcher." The response is that COPPER-LINE is not tracking secrecy or general information flow. It tracks positive authority for a DMP to dereference a source word, with a clean-since-commit lifecycle. RCP then applies the same authority rule recursively: a prefetched line can carry identity and context, but not authority.

A second risk is: "This is just CHERI/MTE." The response is that CHERI/MTE protect architectural pointer or memory safety. COPPER-LINE protects microarchitectural DMP dereference in ordinary pointer systems and does not require capability pointers.

A third risk is: "The results are synthetic." This is still partly valid, but narrower than before. The ARM/AArch64 gem5 integration reduces simulator risk and shows MSHR-level behavior, and the ARM64 full-system path now boots Linux, runs native AArch64 timing ROIs, runs official GAPBS AArch64 binaries, runs public Olden pointer-intensive binaries, runs compiler-authored C and GAPBS-inspired graph-kernel suites, and attaches COPPER in the cache hierarchy. However, COPPER's strongest speedup workloads remain controlled pointer-heavy binaries, while official GAPBS is mainly a safety/control result. Public Olden gives a stronger external workload point, but conventional address-correlation prefetchers such as DCPT and SPP are faster there. A top-conference version still needs broader Linux application traces or full-system workloads with naturally pointer-heavy data structures, known DMP attack patterns, and an evaluation that cleanly separates conventional address-stream performance from content-derived DMP safety.

A fourth risk is: "First-use prefetching is lost." This is true by design. COPPER-LINE only enables DMP dereference after proof. The paper should frame this as a security/performance tradeoff and explore warmup mechanisms that do not violate the invariant.

A fifth risk is: "PEB is just epoch invalidation." That criticism is fair if PEB is presented alone. The paper should present PEB as a boundary hook for COPPER's proof authority: it prevents pre-boundary committed-source proofs from authorizing post-boundary DMP issue, and the evidence is the fake-only control, multi-seed heap ROI, official GAPBS PEB run, and small RTL/synthesis result.

11. Conclusion

COPPER reframes DMP safety around authority. A data-memory-dependent prefetcher should not dereference data merely because it looks like an address. It should dereference only source words that committed execution has already proven to be pointer sources, that remain clean, and that remain bound to the same address-space authority. Trace simulation shows that this invariant blocks modeled Augury/GoFetch-style unsafe activation while retaining meaningful pointer-prefetch speedup. The graph-style traces add two useful controls: source-only provenance is not enough after rewrites, and an edge-exact proof ledger can hit a graph-scan capacity cliff. COPPER epoch/value provenance blocks stale rewritten-edge issue, while CLPD compresses retained source proof by cache line without reauthorizing data-at-rest, unproven-edge, or stale-slot dereferences; the 4,320-run kernel sensitivity sweep keeps COPPER unsafe modeled prefetches at zero while exposing both naive-DMP and source-only failures. The newest full-system heap ROI shows CLPD-64K improving three AArch64/Linux heap layouts, and PEB closes the fake-pointer-only warm-state leakage without losing the measured pointer-workload benefit. Official AArch64 GAPBS now runs cleanly as a public control suite, though it is not pointer-heavy. Public Olden adds pointer-intensive full-system coverage: COPPER CLPD-64K+PEB cuts naive DMP's CTLW misses substantially and preserves zero translation faults, but DCPT and SPP are faster conventional address-correlation prefetchers on the same Olden runs. That result sharpens rather than invalidates the claim: COPPER is not the fastest universal prefetcher; it is an authority mechanism for safely admitting content-derived pointer prefetches. CLPD's compact and SRAM-style RTL now pass directed/randomized XSIM checks, the full 64K SRAM directory synthesizes and routes out-of-context on Artix-7 200T, and PEB synthesizes as a small per-domain epoch/token block. ARM gem5 results show that CPTQ and recursive carried provenance convert many demand-visible MSHR misses into prefetch-origin misses on permuted and random pointer chains. Full-system Linux/AArch64 testing adds three lessons: source proofs must include address-space binding, cross-page recursive targets need committed target-line witnesses rather than fresh speculative translation from prefetched data, and DMP proof state needs an explicit provenance epoch boundary.

The project is now strong enough for a serious workshop or focused conference-style submission draft and closer to a top-conference artifact than before because the AArch64 result survives Minor/O3 sensitivity with bounded traffic overhead, the stale-source proof path has a named RTL mechanism, ARM64 full-system Linux now runs bracketed native AArch64 pointer, graph-gather, compiled C, official GAPBS, fake-only, heap-pointer, SQLite, Lua, Duktape, and yyjson timing ROIs with COPPER in the L1D cache path, a repeated medium/stress SQLite/Lua/Duktape seed portfolio supports public-engine layout stability, GAPBS-generated topology replay exposes and closes a proof-capacity limitation with CLPD, the expanded GAPBS-style sensitivity sweep tests the invariant across graph kernels and hardware knobs, the new traffic/pollution scorecard quantifies side effects rather than assuming them away, the CLPD directory has compact and scalable RTL evidence, PEB removes a real warm-state fake-data caveat, bounded checkers tie PASB/CTLW/terminal rules to explicit counterexamples, the OoO-LSQ proof-contract checker ties proof creation to retirement and source-tag freshness, the TLB/coherence contract checker and matching RTL filter tie target-witness freshness to remap/TLBI/permission/revocation behavior, SARI and CS-SARI add RTL, composition-checker, sensitivity-sweep, and workload-derived proxy evidence for SoC/coherence revocation into COPPER metadata, and a generated coverage matrix maps ten modeled unsafe classes to local evidence. It is still not honest to guarantee top-conference acceptance; the remaining burden is broader pointer-rich application evidence and a production-grade end-to-end integration proof.

References

1. Augury: Using Data Memory-Dependent Prefetchers to Leak Data at Rest. https://www.prefetchers.info/augury.pdf

2. GoFetch: Breaking Constant-Time Cryptographic Implementations Using Data Memory-Dependent Prefetchers. https://gofetch.fail/

3. SplittingSecrets: A Compiler-Based Defense for Preventing Data Memory-Dependent Prefetcher Side-Channels. https://arxiv.org/abs/2601.12270

4. PreFence: A Scheduling-Aware Defense Against Prefetching-Based Side-Channel Attacks. https://arxiv.org/abs/2410.00452

5. PhantomFetch: Obfuscating Loads against Prefetcher Side-Channel Attacks. https://arxiv.org/abs/2511.05110

6. ICP: Exploiting Instruction Correlation for Prefetching Irregular Memory Accesses. https://arxiv.org/abs/2605.15645

7. DX100: A Programmable Data Access Accelerator for Indirection. https://arxiv.org/abs/2505.23073

8. Pointer-Chase Prefetcher for Linked Data Structures. https://arxiv.org/abs/1801.08088

9. SafeSpec: Banishing the Spectre of a Meltdown with Leakage-Free Speculation. https://arxiv.org/abs/1806.05179

10. BliMe: Verifiably Secure Outsourced Computation with Hardware-Enforced Taint Tracking. https://arxiv.org/abs/2204.09649

11. HardTaint: Production-Run Dynamic Taint Analysis via Selective Hardware Tracing. https://arxiv.org/abs/2402.17241

12. PICASSO: Scaling CHERI Use-After-Free Protection to Millions of Allocations using Colored Capabilities. https://arxiv.org/abs/2602.09131

13. ARM MTE Performance in Practice. https://arxiv.org/abs/2601.11786

14. Revelator: Rapid Data Fetching via OS-Driven Hash-based Speculative Address Translation. https://arxiv.org/abs/2508.02007

15. ChampSim. https://github.com/ChampSim/ChampSim

16. GAP Benchmark Suite. https://github.com/sbeamer/gapbs

17. Intel Data Dependent Prefetcher guidance. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/data-dependent-prefetcher.html

18. Self-invalidating TLB Entries. https://www.csa.iisc.ac.in/~arkapravab/papers/pact2017_final_version.pdf

19. ecoTLB: Eventually Consistent TLBs. https://www.cs.yale.edu/homes/abhishek/kumar-taco20.pdf